

Κατάλογος Models

Τελευταία ενημέρωση: 2026-06-29

Αυτό το αρχείο τεκμηριώνει τα model kinds που είναι διαθέσιμα μέσω του `MODEL_REGISTRY` στο `src/models/registry.py`.

Τα models είναι το στάδιο που μαθαίνει σχέση ανάμεσα σε causal features και ένα target. Το output τους δεν έχει πάντα την ίδια σημασία. Ένα classifier παράγει probability, ένας regressor/forecaster παράγει expected return ή volatility, ένα sequence model μπορεί να παράγει embeddings και ένα RL policy παράγει actions ή portfolio weights. Για αυτό το catalog εξηγεί:

- τι μαθαίνει κάθε model,
- τι πληροφορία δίνουν οι τιμές που παράγει,
- πότε είναι κατάλληλο,
- πώς να διαβάζεις ένα πρακτικό παράδειγμα.

Πώς διαβάζεις model outputs

Συνηθισμένες output στήλες:

- `pred_prob`: πιθανότητα θετικού class ή πιθανότητα επιτυχίας candidate.
- `pred_ret`: forecast μελλοντικής απόδοσης ή target value.
- `pred_vol`: forecast μεταβλητότητας/risk.
- `pred_is_oos`: flag ότι η πρόβλεψη είναι out-of-sample για το row.
- `event_emb_*`: learned event/sequence embeddings από transformer encoder.
- `signal_rl`: σήμα που προκύπτει από RL policy.
- `action_rl`: raw action ή discrete action id από RL policy.

Γενική ερμηνεία:

- `pred_prob` κοντά στο `0.5`: αβεβαιότητα για binary classifier.
- `pred_prob` υψηλό, π.χ. `0.70`: το model θεωρεί πιο πιθανό το positive label ή την επιτυχία του candidate.
- `pred_prob` χαμηλό, π.χ. `0.30`: το positive label θεωρείται λιγότερο πιθανό.
- `pred_ret > 0`: θετικό expected future return στην κλίμακα του target.
- `pred_ret < 0`: αρνητικό expected future return.
- `pred_vol` υψηλό: αυξημένο forecast risk, όχι απαραίτητα short signal.
- `pred_is_oos = 1`: η πρόβλεψη μπορεί να χρησιμοποιηθεί για signal/backtest diagnostics.
- `pred_is_oos = 0`: training/in-sample περιοχή, χρήσιμη για fit diagnostics αλλά όχι για trading signal.

Σημαντικός κανόνας: signal builders πρέπει να χρησιμοποιούν out-of-sample predictions. Αν ένα signal βασιστεί σε in-sample `pred_prob` ή `pred_ret`, το backtest έχει leakage.

Κατηγορίες

Κατηγορία	Models	Τι μαθαίνουν
Tabular classifiers	<code>logistic_regression_clf</code> , <code>elastic_net_clf</code> , <code>lightgbm_clf</code> , <code>xgboost_clf</code>	Πιθανότητα binary label ή meta-label από tabular features.
Sequence / event classifier	<code>event_transformer_encoder</code>	Πιθανότητα και embeddings από rolling event sequences.
Return forecasters / regressors	<code>lightgbm_regressor</code> , <code>sarimax_forecaster</code> , <code>lstm_forecaster</code> , <code>patchtst_forecaster</code> , <code>tft_forecaster</code>	Μελλοντική απόδοση ή continuous target.
Foundation zero-shot forecasters	<code>chronos_bolt_forecaster</code> , <code>chronos_2_forecaster</code> , <code>timesfm_2p5_200m_forecaster</code> , <code>timesfm_1p0_200m_forecaster</code>	Inference-only forecast από causal price/returns context.
Volatility / risk forecaster	<code>garch_forecaster</code>	Κυρίως forecast volatility/risk από return dynamics.
Feature discovery	<code>tsfresh_extrema_feature_discovery</code>	PIT-safe tsfresh features και relevance για future extrema labels.
Single-asset RL	<code>ppo_agent</code> , <code>dqn_agent</code>	Policy για actions σε ένα asset.
Portfolio RL	<code>ppo_portfolio_agent</code> , <code>dqn_portfolio_agent</code>	Policy για allocation/actions σε πολλά assets.
Market-making research	<code>market_making_moment</code> config	Research-only MOMENT quote filter για fee-adjusted markout, εκτός live/demo execution.

Κοινή δομή YAML για models

Τα περισσότερα model entries δέχονται την ίδια εξωτερική δομή. Οι επιμέρους αλγόριθμοι διαβάζουν δικά τους `params`, αλλά τα παρακάτω πεδία έχουν κοινή σημασία:

- **kind**: το registry name του model. Παράδειγμα: `lightgbm_clf`. Αν το `kind` είναι λάθος ή δεν υπάρχει στο `MODEL_REGISTRY`, το experiment δεν πρέπει να προχωρήσει σιωπηλά.
- **target**: ο ορισμός του label/continuous target που θα χτιστεί πριν το fit. Για classifier συνήθως παράγει `label`. Για forecaster παράγει continuous forward return ή path-dependent outcome. Τα target output columns δεν πρέπει να ξανααπαίτουν στα features.
- **feature_cols**: explicit λίστα στηλών που θα χρησιμοποιηθούν ως inputs. Είναι η πιο ελεγχόμενη επιλογή όταν θέλεις auditability.
- **feature_selectors**: κανόνες επιλογής στηλών όταν δεν θέλεις να γράψεις κάθε feature με το χέρι. Χρησιμοποιείται για prefixes/families/profiles. Θέλει προσοχή ώστε να μη μαζέψει target, signal ή prediction columns.

- **split**: temporal split policy. Συνήθεις τιμές: `time`, `walk_forward`, `purged`. Η έννοια είναι πάντα χρονική, όχι random shuffle. Το train window κόβεται όπου χρειάζεται ώστε το target horizon να μη διαρρέει στο test.
- **params**: estimator-specific hyperparameters. Αυτά περνάνε στον underlying estimator ή στο neural/RL trainer, με defaults που αναφέρονται παρακάτω.
- **runtime**: reproducibility/threading controls. Κοινά πεδία: `seed`, `deterministic`, `repro_mode`, `threads`.
- **preprocessing**: train-only preprocessing για classifiers. Υποστηρίζει `scaler`: `none`, `standard` ή `robust`. Το scaler fit γίνεται μέσα σε κάθε train fold και εφαρμόζεται στο αντίστοιχο test fold.
- **calibration**: probability calibration για classifiers. Υποστηρίζεται `method`: `none` ή `sigmoid`, με `fraction` και `min_rows`.
- **pred_prob_col**, **pred_ret_col**, **pred_raw_prob_col**, **pred_is_oos_col**, **signal_col**, **action_col**: ονόματα output columns. Η αλλαγή ονόματος είναι χρήσιμη όταν τρέχεις πολλά models στο ίδιο dataframe, αλλά πρέπει να ενημερωθούν και τα downstream signals.
- **overlay**: optional πρόσθετο fold-level predictor, σήμερα κυρίως `garch`, ώστε ένα classifier/forecaster να γράφει και risk columns όπως `pred_vol`.

Παράδειγμα κοινής δομής:

```
model:
  kind: lightgbm_clf
  target:
    kind: forward_return
    horizon: 8
    threshold: 0.0
    label_col: label
    fwd_col: fwd_return
  feature_selectors:
    - startswith: ["ema_", "atr_", "rsi_"]
  split:
    method: walk_forward
    train_size: 0.70
    test_size: 0.10
  runtime:
    seed: 7
    deterministic: true
    threads: 1
  params:
    n_estimators: 300
    learning_rate: 0.05
```

Ερμηνεία: το target κοιτάει 8 bars μπροστά για να φτιάξει label, αλλά τα features στο row `t` πρέπει να είναι διαθέσιμα στο `t`. Το model fit γίνεται σε train folds, γράφει predictions σε OOS test folds και σημαδεύει αυτά τα rows με `pred_is_oos = true`.

Κοινές παράμετροι και γιατί έχουν σημασία

target

- **kind**: επιλέγει target builder. Για binary classification, π.χ. **forward_return** ή **directional_triple_barrier**. Για regression, **future_return_regression**.
- **horizon / horizon_bars**: πόσα bars μπροστά κοιτάει το label. Μεγαλύτερο horizon μειώνει τον θόρυβο ενός bar, αλλά αυξάνει overlap και απαιτεί πιο αυστηρό train trimming.
- **threshold**: το όριο πάνω από το οποίο ένα forward return γίνεται positive label. Παράδειγμα: **threshold: 0.001** σημαίνει ότι +0.05% δεν είναι επιτυχία, ενώ +0.15% είναι.
- **label_col**: το όνομα της στήλης label. Αν το αλλάξεις, το model θα εκπαιδευτεί στο νέο όνομα και τα diagnostics θα το αναφέρουν εκεί.
- **fwd_col**: το όνομα της raw/continuous forward outcome στήλης.
- **candidate_col**: περιορίζει training/prediction σε candidate rows. Σε meta-labeling αυτό είναι κρίσιμο: το model δεν απαντά "θα ανέβει η αγορά;", αλλά "αξιίζει αυτό το candidate trade;".
- **side_col**: δίνει long/short πλευρά σε directional/meta targets. Χωρίς σωστό **side_col**, ένα meta model μπορεί να μπερδέψει επιτυχές short με αρνητική αγορά αντί με επιτυχές trade.

feature_cols και feature_selectors

- **feature_cols**: explicit columns. Παράδειγμα: **["close_logret_1", "atr_pct_14", "rsi_14"]**.
- **feature_selectors**: κανόνες επιλογής. Παράδειγμα: - **startswith: ["ema_", "stoch_"]**. Είναι πρακτικό για μεγάλα feature sets, αλλά πρέπει να ελέγχεις το resolved feature list στα metadata.
- **minimum_expected_features**: σε forecasters μπορεί να χρησιμοποιηθεί ως sanity check για να μη γίνει fit με πολύ λίγα features μετά από filters.

split

- **method: time**: ένα χρονικό train/test split.
- **method: walk_forward**: πολλά chronological folds. Προτιμάται για trading diagnostics γιατί δείχνει stability ανά περίοδο.
- **method: purged**: split με purging/embargo semantics όπου υποστηρίζεται από τα split utilities.
- **train_size, test_size, n_splits**: ελέγχουν μέγεθος folds. Αν το test είναι πολύ μικρό, τα OOS metrics έχουν υψηλή variance.
- **target_horizon**: δεν το περνάς συνήθως στο split άμεσα. Προκύπτει από το target και χρησιμοποιείται για να κοπούν train rows που θα έβλεπαν μέσα στο test horizon.

runtime

- **seed**: random seed για sklearn/LightGBM/XGBoost/PyTorch όπου εφαρμόζεται. Παράδειγμα: **seed: 7** κάνει τα ίδια folds και ίδια stochastic initialization πιο αναπαραγώγιμα.
- **deterministic**: ζητά deterministic behavior όπου το backend το υποστηρίζει. Σε neural models μπορεί να μειώσει ταχύτητα αλλά αυξάνει reproducibility.
- **repro_mode: strict** βάζει πιο συντηρητικές ρυθμίσεις, όπως default single-thread όπου δεν ορίζεται **threads**. **relaxed** είναι πιο πρακτικό για γρήγορα πειράματα, αλλά λιγότερο αυστηρό.
- **threads**: αριθμός threads. **threads: 1** είναι πιο reproducible. Περισσότερα threads είναι γρηγορότερα αλλά μπορεί να αλλάξουν floating-point ordering.

preprocessing

- **scaler: none**: δεν κλιμακώνει features. Συνήθως OK για tree models.

- **scaler: standard**: αφαιρεί train mean και διαίρει με train std ανά fold. Είναι default για **logistic_regression_clf** και **elastic_net_clf**.
- **scaler: robust**: χρησιμοποιεί robust scaling, πιο ανθεκτικό σε outliers.
- Σημαντικό: το scaler fit γίνεται μόνο στο train fold. Αν κάνεις scaling σε όλο το dataframe πριν το split, εισάγεις leakage.

calibration

- **method: none**: κρατά raw estimator probability.
- **method: sigmoid**: κρατά τελευταίο κομμάτι του train fold για sigmoid calibration. Χρήσιμο όταν probabilities είναι overconfident.
- **fraction**: ποιο ποσοστό του train fold πάει στο calibration window. Παράδειγμα: **0.20** σημαίνει 80% estimator fit και 20% calibration.
- **min_rows**: ελάχιστα calibration rows. Αν είναι πολύ χαμηλό, το calibrator μπορεί να μάθει θόρυβο αντί για calibration.
- **pred_raw_prob_col**: αν δοθεί ή αν υπάρχει calibration, κρατά raw probability πριν την calibration.

Παράμετροι output columns

- **pred_prob_col**: όνομα probability output για classifiers και derived probability για forecasters.
- **pred_ret_col**: όνομα continuous forecast output για forecasters.
- **pred_is_oos_col**: boolean στήλη που δείχνει ποια rows είναι out-of-sample.
- **signal_col**: RL output exposure/action-translated signal. Default: **signal_rl**.
- **action_col**: raw RL action id/value για single-asset RL. Default: **action_rl**.
- **prediction_col**: σε παλιότερα configs μπορεί να εμφανίζεται ως γενικό όνομα, αλλά τα τρέχοντα wrappers χρησιμοποιούν τα explicit names παραπάνω.

Παράμετροι tabular classifiers

Οι classifiers χρησιμοποιούν κοινό anti-leakage pipeline:

- χτίζουν target από το **target**,
- επιλέγουν features με **feature_cols** ή **feature_selectors**,
- κάνουν temporal splits,
- κόβουν train rows που θα έβλεπαν μέσα στο test horizon,
- κάνουν fit μόνο σε complete train rows,
- γράφουν **pred_prob** μόνο σε complete/candidate OOS test rows.

Κοινές classifier παράμετροι

- **target**: binary target definition. Αν είναι candidate-based, οι προβλέψεις γράφονται μόνο σε candidate rows.
- **feature_cols** / **feature_selectors**: inputs στο estimator.
- **split.method**: **time**, **walk_forward** ή **purged**.
- **preprocessing.scaler**: **none**, **standard**, **robust**.
- **calibration.method**: **none** ή **sigmoid**.
- **pred_prob_col**: default **pred_prob**.
- **pred_raw_prob_col**: default **pred_prob_raw** όταν χρειάζεται raw probability.
- **pred_is_oos_col**: default **pred_is_oos**.

- `params`: estimator hyperparameters.

Παράμετροι `logistic_regression_clf`

- `params.max_iter`: default `1000`. Μέγιστος αριθμός optimizer iterations. Αν βλέπεις convergence warnings, αύξησέ το, π.χ. `3000`.
- `params.solver`: default `lbfgs`. Optimizer του sklearn logistic regression. Για απλό L2 logistic είναι σταθερή επιλογή.
- `params.C`: inverse regularization strength του sklearn, αν το δηλώσεις. Μικρότερο `C` σημαίνει πιο έντονο shrinkage. Παράδειγμα: `C: 0.2` τιμωρεί μεγάλα coefficients περισσότερο από `C: 1.0`.
- `params.class_weight`: π.χ. `balanced`. Χρήσιμο σε imbalanced labels, αλλά μπορεί να αλλάξει calibration.
- `preprocessing.scaler`: default `standard`. Χρειάζεται γιατί το logistic regression είναι ευαίσθητο στην κλίμακα των features.

Παράμετροι `elastic_net_clf`

- `params.penalty`: πρέπει να είναι `elasticnet`. Αν δοθεί κάτι άλλο, ο κώδικας σηκώνει error.
- `params.solver`: πρέπει να είναι `saga`, γιατί αυτός υποστηρίζει elastic-net logistic regression στο sklearn.
- `params.l1_ratio`: default `0.5`. `0.0` μοιάζει με L2, `1.0` μοιάζει με L1. Με `0.8` θα μηδενίσει πιο πολλά coefficients από `0.2`.
- `params.C`: default `1.0`. Μικρότερο `C` κάνει πιο επιθετικό regularization.
- `params.max_iter`: default `2000`. Συνήθως χρειάζεται περισσότερο από το απλό logistic λόγω `saga`.
- `preprocessing.scaler`: default `standard`. Χωρίς scaling, το L1 κομμάτι τιμωρεί δυσανάλογα features με μεγάλη αριθμητική κλίμακα.

Παράμετροι `lightgbm_clf`

Τα `params` περνάνε σε `LGBMClassifier`. Τα πιο συνηθισμένα:

- `n_estimators`: αριθμός boosting trees. Περισσότερα trees αυξάνουν capacity. Παράδειγμα: `600` μπορεί να πιάσει πιο λεπτές σχέσεις από `100`, αλλά θέλει αυστηρό OOS έλεγχο.
- `learning_rate`: βήμα κάθε tree. Μικρότερο learning rate συνήθως απαιτεί περισσότερα trees.
- `num_leaves`: μέγιστος αριθμός leaves ανά tree. Μεγάλο `num_leaves` αυξάνει nonlinear capacity και overfit risk.
- `max_depth`: μέγιστο βάθος tree. `-1` σημαίνει χωρίς explicit limit.
- `subsample`: ποσοστό rows ανά boosting iteration. Κάτω από `1.0` προσθέτει stochastic regularization.
- `colsample_bytree`: ποσοστό features ανά tree. Χρήσιμο όταν έχεις πολλά correlated features.
- `min_child_samples`: ελάχιστα rows σε leaf. Μεγαλύτερη τιμή κάνει πιο συντηρητικά leaves.
- `random_state` / `seed`: reproducibility. Αν δεν δοθούν, το runtime resolution βάζει seed από `runtime.seed`.
- `n_jobs`: threads. Σε `strict` mode γίνεται συνήθως `1` αν δεν οριστεί.

Παράμετροι `xgboost_clf`

Τα `params` περνάνε σε `XGBClassifier`, με normalization από wrapper:

- `objective`: default `binary:logistic`. Παράγει probability για positive class.

- `eval_metric`: default `logloss`. Metric που χρησιμοποιεί το XGBoost internally.
- `tree_method`: default `hist`. Histogram-based training, συνήθως γρήγορο και σταθερό.
- `n_estimators`: αριθμός trees.
- `max_depth`: βάθος trees. Μεγαλύτερο βάθος πιάνει interactions αλλά αυξάνει overfit.
- `learning_rate`: boosting step size.
- `subsample`: row sampling ratio.
- `colsample_bytree`: feature sampling ratio.
- `reg_alpha`: L1 regularization.
- `reg_lambda`: L2 regularization.
- `seed` / `random_state`: reproducibility.
- `num_leaves` και `min_child_samples`: αφαιρούνται αν περαστούν κατά λάθος, γιατί είναι LightGBM-style params και όχι XGBoost params.

Παράμετροι sequence/event classifier

`event_transformer_encoder`

Απαιτεί candidate-based target, πρακτικά target με `candidate_col`. Δεν είναι γενικός classifier για κάθε row.

- `target.candidate_col`: υποχρεωτικό μέσω target metadata. Ορίζει σε ποια rows υπάρχουν events για training/prediction.
- `feature_cols` / `feature_selectors`: numeric feature inputs για κάθε bar του sequence.
- `pred_prob_col`: optional. Αν δοθεί, γράφει classifier head probability. Αν δεν δοθεί, γράφει μόνο embeddings.
- `pred_is_oos_col`: default `pred_is_oos`.
- `params.lookback`: default 48. Πόσα trailing bars μπαίνουν στο sequence που τελειώνει στο event timestamp. Πρέπει να είναι > 1 .
- `params.hidden_dim`: default 32. Transformer hidden width. Πρέπει να διαιρείται από `num_heads`.
- `params.num_heads`: default 4. Attention heads. Περισσότερα heads αυξάνουν capacity αλλά απαιτούν μεγαλύτερο `hidden_dim`.
- `params.num_layers`: default 2. Πόσα TransformerEncoder layers.
- `params.dropout`: default 0.1. Regularization μέσα στο network. Πρέπει να είναι στο $[0, 1)$.
- `params.epochs`: default 8. Training passes πάνω στα train event samples.
- `params.batch_size`: default 64. Batch size στο DataLoader. Ο κώδικας δεν αφήνει effective batch κάτω από 8.
- `params.learning_rate`: default $1e-3$. AdamW learning rate.
- `params.weight_decay`: default $1e-4$. L2-style regularization στο AdamW.
- `params.embedding_dim`: default ίσο με `hidden_dim`. Πόσες `event_emb_*` στήλες θα γραφτούν.
- `params.embedding_prefix`: default `event_emb`. Με `embedding_dim: 4` γράφει `event_emb_00` έως `event_emb_03`.
- `params.min_train_samples`: default 32. Ελάχιστα sequence samples ανά fold. Αν δεν υπάρχουν, το fold αποτυγχάνει αντί να εκπαιδεύσει άχρηστο model.

Παράδειγμα:

```
model:
  kind: event_transformer_encoder
```



```

target:
  kind: directional_triple_barrier
  candidate_col: signal_candidate
  side_col: signal_side
  horizon: 16
feature_selectors:
  - startswith: ["ema_", "atr_", "stoch_", "rsi_"]
pred_prob_col: event_pred_prob
params:
  lookback: 32
  hidden_dim: 64
  num_heads: 4
  embedding_dim: 16

```

Ερμηνεία: κάθε candidate παίρνει 32-bar ιστορικό context. Το model γράφει πιθανότητα επιτυχίας candidate και 16 embedding columns που μπορούν να χρησιμοποιηθούν σε diagnostics ή second-stage models.

Παράμετροι forecasters/regressors

Οι forecasters χρησιμοποιούν κοινό pipeline που γράφει:

- `pred_ret_col`, default `pred_ret`,
- `pred_prob_col`, default `pred_prob`, ως μετατροπή forecast σε probability γύρω από το target threshold,
- `pred_is_oos_col`, default `pred_is_oos`,
- optional `pred_vol` ή quantile columns, ανά model.

Κοινές forecaster παράμετροι

- `target`: continuous target. Συνήθως `future_return_regression` ή `forward_return`.
- `use_features`: default `true`. Αν `false`, statistical forecaster μπορεί να αγνοήσει exogenous features.
- `feature_cols` / `feature_selectors`: exogenous inputs για models που τα απαιτούν.
- `pred_ret_col`: output forecast column.
- `pred_prob_col`: derived probability column. Δεν είναι classifier probability από cross-entropy, αλλά mapping του forecast σε πιθανότητα θετικού outcome.
- `pred_is_oos_col`: OOS flag.
- `diagnostics`: optional diagnostics config, π.χ. LightGBM SHAP.

Παράμετροι `lightgbm_regressor`

Προεπιλεγμένες estimator παράμετροι από το wrapper:

- `n_estimators`: default 300. Αριθμός boosting trees.
- `learning_rate`: default 0.03. Μικρότερο από classifier default γιατί τα regression forecasts είναι συχνά πιο θορυβώδη.
- `max_depth`: default -1. Χωρίς explicit depth limit.
- `num_leaves`: default 31. Βασικός έλεγχος tree complexity.
- `subsample`: default 1.0. Row sampling.
- `colsample_bytree`: default 1.0. Feature sampling.

- `random_state`: default 7 αν δεν δοθεί από runtime.
- `n_jobs`: default 1. Συντηρητικό για reproducibility.
- `verbosity`: default -1. Μειώνει LightGBM logging.
- `minimum_expected_features`: internal/sanity parameter, δεν περνιέται στον estimator.

Παράδειγμα: αν `pred_ret = 0.004` και το target είναι raw return, η πρόβλεψη είναι +0.4%. Αν το target είναι normalized by volatility, τότε `0.004` δεν σημαίνει +0.4%, αλλά 0.004 μονάδες του normalized target.

Παράμετροι `sarimax_forecaster`

- `params.order`: default (1, 0, 1). ARIMA order (p, d, q). `p` είναι AR lags, `d` differencing, `q` moving-average terms.
- `params.seasonal_order`: default (0, 0, 0, 0). Seasonal (P, D, Q, s). Παράδειγμα: σε intraday data με ημερήσιο κύκλο 48 bars, seasonal period μπορεί να είναι 48, αν έχει νόημα και αρκετά δεδομένα.
- `params.trend`: default c. Constant trend/intercept.
- `params.enforce_stationarity`: default false. Αν true, αναγκάζει stationarity constraints.
- `params.enforce_invertibility`: default false. Αν true, αναγκάζει invertibility constraints.
- `params.maxiter`: default 200. Μέγιστες optimizer iterations.
- `params.use_exog`: default true. Αν true, χρησιμοποιεί resolved feature columns ως exogenous regressors. Αν υπάρχουν missing exog στο test fold, αποτυγχάνει για να μην ευθυγραμμίσει λάθος forecasts.
- `params.allow_fallback`: default true. Αν το SARIMAX fit αποτύχει, γυρίζει σε fallback mean/variance forecast αντί να ρίξει όλο το experiment.

Παράδειγμα: `order: [1, 0, 1]` σημαίνει ότι το forecast χρησιμοποιεί ένα AR lag και ένα MA term χωρίς differencing. Αν το forecast γίνεται σχεδόν σταθερό γύρω από το train mean, μάλλον δεν υπάρχει αρκετή autocorrelation structure.

Παράμετροι `garch_forecaster`

- `params.returns_input_col`: default `close_ret` ή `returns_col` αν περασθεί. Είναι η return σειρά πάνω στην οποία fit γίνεται το GARCH.
- `target.price_col`: default `close` για fallback υπολογισμό returns όταν δεν υπάρχει `returns_input_col`.
- `params.mean_model`: default `constant`. Επιτρεπτά: `zero`, `constant`, `ar1`.
- `mean_model: zero`: forecast return mean = 0. Κατάλληλο όταν θες μόνο volatility forecast.
- `mean_model: constant`: forecast return mean = train mean.
- `mean_model: ar1`: προσθέτει AR(1) mean term με clipped `phi`.
- GARCH variance params στο metadata: `omega`, `alpha`, `beta`. Υψηλό `alpha` αντιδρά έντονα στο τελευταίο shock. Υψηλό `beta` κάνει τη volatility πιο persistent.

Παράδειγμα: αν `pred_vol` ανεβαίνει από `0.006` σε `0.018`, το model βλέπει τριπλάσιο expected one-step volatility. Αυτό δεν είναι long/short edge, αλλά risk state για sizing, filters ή `forecast_vol_adjusted`.

Κοινές παράμετροι neural forecasters

Ισχύουν για `lstm_forecaster`, `patchtst_forecaster`, `tft_forecaster`, με μικρές διαφορές στα defaults:

- `params.lookback`: πόσα trailing bars μπαίνουν στο sequence.
- `params.hidden_dim`: latent width του network.
- `params.num_layers`: αριθμός recurrent/transformer layers.
- `params.num_heads`: attention heads σε transformer-style models. Το `hidden_dim` πρέπει να διαιρείται από το `num_heads`.
- `params.dropout`: regularization. Πρέπει να είναι στο `[0, 1)`.
- `params.epochs`: training epochs ανά fold.
- `params.batch_size`: batch size.
- `params.learning_rate`: AdamW learning rate.
- `params.weight_decay`: AdamW regularization.
- `params.scale_target`: default `true`. Κάνει target scaling μέσα στο train fold και inverse-transform στις predictions. Βοηθά neural optimization χωρίς να αλλάζει την τελική κλίμακα του `pred_ret`.
- `params.quantiles`: quantile forecasts. Όταν υπάρχουν, γράφει `pred_q10`, `pred_q50`, `pred_q90` κ.λπ. και χρησιμοποιεί median quantile ως `pred_ret`.

Παράμετροι `lstm_forecaster`

- `lookback`: default 48.
- `hidden_dim`: default 32.
- `num_layers`: default 2.
- `dropout`: default 0.1; εφαρμόζεται στο LSTM μόνο όταν `num_layers > 1`.
- `epochs`: default 12.
- `batch_size`: default 64.
- `learning_rate`: default 1e-3.
- `weight_decay`: default 1e-4.
- `quantiles`: default κενό. Αν δεν δοθεί, κάνει MSE regression.
- `scale_target`: default `true`.

Παράμετροι `patchtst_forecaster`

- `lookback`: default 64.
- `patch_len`: default 8. Πόσα bars περιέχει κάθε patch. Πρέπει να είναι `> 1` και `<= lookback`.
- `patch_stride`: default 4. Απόσταση μεταξύ διαδοχικών patches.
- `hidden_dim`: default 64.
- `num_heads`: default 4.
- `num_layers`: default 2.
- `dropout`: default 0.1.
- `epochs`: default 12.
- `batch_size`: default 64.
- `learning_rate`: default 1e-3.
- `weight_decay`: default 1e-4.
- `quantiles`: default `[0.1, 0.5, 0.9]`.
- `scale_target`: default `true`.

Παράδειγμα: με `lookback: 64`, `patch_len: 8`, `patch_stride: 4`, το model βλέπει overlapping 8-bar blocks. Έτσι μπορεί να μάθει pattern τύπου compression -> expansion που δεν χωράει σε ένα μόνο row.

Παράμετροι `tft_forecaster`

- `lookback`: default 32.
- `hidden_dim`: default 32.
- `num_heads`: default 4.
- `num_layers`: default 2.
- `dropout`: default 0.1.
- `epochs`: default 20.
- `batch_size`: default 64.
- `learning_rate`: default 1e-3.
- `weight_decay`: default 1e-4.
- `scale_target`: default true.
- `min_train_samples`: default 32. Κάτω από αυτό το fold αποτυγχάνει.
- `quantiles`: default [0.1, 0.5, 0.9], πρέπει να είναι μοναδικά και μέσα στο (0, 1).

Παράμετροι foundation forecasters

Ισχύουν για `chronos_bolt_forecaster`, `chronos_2_forecaster`, `timesfm_2p5_200m_forecaster` και `timesfm_1p0_200m_forecaster`.

- `params.source_col`: observed causal σειρά που δίνεται στο foundation model. Default είναι το `model.target.price_col` ή `close`.
- `params.source_kind`: `price` ή `returns`. Αν λείπει, γίνεται inference από το όνομα του `source_col`.
- `params.source_returns_type`: `simple` ή `log` όταν το source είναι returns.
- `params.model_id`: checkpoint id. Defaults: `amazon/chronos-bolt-tiny`, `amazon/chronos-2`, `google/timesfm-2.5-200m-pytorch`, `google/timesfm-1.0-200m-pytorch`.
- `params.lookback`: maximum context bars που περνάνε στο model. Default 256.
- `params.min_context`: minimum finite context bars για να βγει prediction. Default 16.
- `params.prediction_length`: forecast horizon που ζητάς από το foundation model. Default είναι το target horizon και πρέπει να είναι $\geq$ target horizon.
- `params.quantiles`: default [0.1, 0.5, 0.9]. Γράφει `pred_q10`, `pred_q50`, `pred_q90` όταν το backend επιστρέφει quantiles, και `pred_vol` από το low/high spread.

Παράδειγμα:

```
model:
  kind: chronos_bolt_forecaster
  target:
    kind: forward_return
    price_col: close
    horizon: 4
  split:
    method: walk_forward
    train_size: 500
    test_size: 100
  params:
    model_id: amazon/chronos-bolt-tiny
    source_col: close
    source_kind: price
    lookback: 256
```

```
min_context: 32
quantiles: [0.1, 0.5, 0.9]
```

Σημείωση: αυτά τα wrappers δεν εκπαιδεύουν model ανά fold. Χρησιμοποιούν μόνο το causal context μέχρι το row `t`, προβλέπουν future price/returns και μετά μετατρέπουν το forecast σε `pred_ret`, ώστε να συγκρίνεται με το υπάρχον `forward_return` ή `future_return_regression` target.

Παράμετροι RL models

Τα RL models δεν εκπαιδεύουν supervised label probability. Χρησιμοποιούν trading environment, returns, costs και execution constraints για να μάθουν policy. Για αυτό τα outputs είναι `signal_rl` και, στα single-asset models, `action_rl`.

Κοινές RL παράμετροι

- `feature_cols` / `feature_selectors`: numeric state features. Απαγορεύονται feature columns που ξεκινούν με `signal_`, `pred_`, `target_`, `action_`, για να μην περάσουν downstream/label πληροφορίες στο state.
- `backtest.returns_col`: default `close_ret`. Η return στήλη που δίνει reward.
- `backtest.returns_type`: default `simple`. Επιτρεπτά `simple` ή `log`. Τα log returns μετατρέπονται σε simple μέσα στο RL pipeline.
- `target.horizon`: default `1` και σήμερα πρέπει να είναι `1`.
- `env.window_size`: default `32`. Πόσα bars state history μπαίνουν στο observation.
- `env.execution_lagBars`: default `1` και σήμερα πρέπει να είναι `1`.
- `env.max_signal_abs` / `env.max_position`: default `1.0`. Μέγιστη απόλυτη έκθεση. Αν δηλωθούν και τα δύο, πρέπει να ταιριάζουν.
- `env.action_space`: `continuous` ή `discrete`. Για PPO default είναι `continuous`, για DQN default/απαίτηση είναι `discrete`.
- `env.position_grid`: default `[-1.0, 0.0, 1.0]` για single-asset discrete actions.
- `env.reward.cost_per_turnover`: κόστος ανά μονάδα turnover.
- `env.reward.slippage_per_turnover`: slippage ανά μονάδα turnover.
- `env.reward.inventory_penalty`: ποινή για διακράτηση exposure.
- `env.reward.drawdown_penalty`: ποινή drawdown στο reward.
- `env.reward.switching_penalty`: ποινή συχνής αλλαγής action.
- `env.min_holdingBars`: ελάχιστα bars πριν αλλάξει θέση.
- `env.action_hysteresis`: μικρές αλλαγές action αγνοούνται για να μειωθεί churn.
- `risk.dd_guard.enabled`: ενεργοποιεί drawdown guard.
- `risk.dd_guard.max_drawdown`: drawdown όριο, default `0.2`.
- `risk.dd_guard.cooloffBars`: default `20`. Bars αναμονής μετά από guard.
- `risk.dd_guard.rearm_drawdown`: optional επίπεδο re-arm.
- `params`: περνάνε στο Stable-Baselines3 trainer, π.χ. `total_timesteps`, `learning_rate`, `gamma`, `batch_size`, ανά algorithm.

ppo_agent

- Χρησιμοποιεί single-asset PPO.

- Με `env.action_space: continuous`, η action μετατρέπεται σε continuous exposure μέσα στο `[-max_signal_abs, max_signal_abs]`.
- Με `env.action_space: discrete`, χρησιμοποιεί `position_grid`, αλλά PPO συνήθως προτιμάται για continuous control.

dqn_agent

- Απαιτεί `env.action_space: discrete`.
- `action_rl` είναι index στο `position_grid`.
- `signal_rl` είναι η πραγματική έκθεση μετά το mapping. Παράδειγμα: `position_grid: [-1, 0, 0.5, 1]` και `action_rl = 2` σημαίνει `signal_rl = 0.5`.

Παράμετροι portfolio RL

- `data_alignment / alignment`: default `inner`, και σήμερα απαιτείται `inner`. Όλα τα assets ευθυγραμμίζονται στο κοινό timestamp intersection.
- `portfolio.asset_groups`: mapping asset -> group. Χρήσιμο για group exposure constraints.
- `portfolio.long_short`: default `true`. Αν `false`, το environment πρέπει να μη δίνει short weights μετά τα constraints.
- `portfolio.gross_target`: default `1.0`. Επιθυμητή συνολική gross exposure.
- `portfolio.constraints.min_weight`: default `-1.0`.
- `portfolio.constraints.max_weight`: default `1.0`.
- `portfolio.constraints.max_gross_leverage`: default `1.0`.
- `portfolio.constraints.target_net_exposure`: default `0.0`.
- `portfolio.constraints.turnover_limit`: optional cap στο turnover.
- `portfolio.constraints.group_max_exposure`: optional group caps, π.χ. `{equity_index: 0.8, metals: 0.4}`.
- `env.action_templates`: για portfolio DQN discrete actions. Πρέπει να έχει shape `[n_actions, n_assets]`.

ppo_portfolio_agent

- Default `env.action_space: continuous`.
- Η policy βγάζει per-asset signals, τα οποία μετά περνάνε από portfolio constraints. Μην αξιολογείς κάθε asset σαν ανεξάρτητο classifier.

dqn_portfolio_agent

- Απαιτεί `env.action_space: discrete`.
- Αν δεν δοθούν `action_templates`, φτιάχνονται defaults: flat, all-long, all-short, και single-asset long/short templates.
- `action_rl` δεν γράφεται per asset όπως στο single-asset wrapper. Το κύριο output είναι per-asset `signal_rl`.

Tabular classifiers

logistic_regression_clf

Τι μαθαίνει:

- Γραμμική σχέση ανάμεσα στα features και την πιθανότητα του positive class.
- Είναι ισχυρό baseline γιατί είναι απλό, ερμηνεύσιμο και δύσκολα υπερπροσαρμόζει σε σχέση με πιο σύνθετα models.

Τι σημαίνουν οι τιμές:

- `pred_prob = 0.65` σημαίνει ότι, σύμφωνα με το γραμμικό μοντέλο, το positive label έχει πιθανότητα περίπου 65%.
- Coefficients με θετικό πρόσημο αυξάνουν το log-odds του positive class όταν το feature μεγαλώνει.
- Coefficients με αρνητικό πρόσημο μειώνουν το log-odds.

Παράδειγμα:

- Αν εκπαιδεύεις σε `forward_return` label με `threshold = 0`, τότε `pred_prob = 0.62` σημαίνει "το μοντέλο εκτιμά 62% πιθανότητα το future return να είναι θετικό". Ένα `probability_threshold` signal μπορεί να κάνει long μόνο πάνω από `0.58`.

Πότε το προτιμάς:

- Για πρώτο classifier baseline.
- Όταν θέλεις πιο καθαρή ερμηνεία feature directions.
- Όταν έχεις πολλά correlated features και θέλεις να δεις αν υπάρχει γραμμικό edge πριν πας σε trees.

`elastic_net_clf`

Τι μαθαίνει:

- Logistic regression με elastic-net regularization.
- Συνδυάζει L1 sparsity και L2 shrinkage μέσω `l1_ratio`.

Τι σημαίνουν οι τιμές:

- `pred_prob` διαβάζεται όπως στο logistic regression.
- Features με coefficient κοντά στο `0` πρακτικά αγνοούνται.
- Μεγαλύτερο L1 μέρος κάνει το μοντέλο πιο επιλεκτικό σε features.

Παράδειγμα:

- Έχεις 400 technical και normalization features. Το `elastic_net_clf` μπορεί να κρατήσει μόνο μικρό subset με μη μηδενικά coefficients. Αν `pred_prob = 0.55`, είναι ήπια θετικό signal, αλλά ίσως όχι αρκετό αν τα costs απαιτούν threshold `0.60`.

Πότε το προτιμάς:

- Όταν έχεις πολλά features και θέλεις regularized feature selection.
- Όταν θες interpretable classifier αλλά το απλό logistic είναι ασταθές.

`lightgbm_clf`

Τι μαθαίνει:

- Nonlinear tree-boosting classifier.

- Μαθαίνει thresholds, interactions και nonlinear σχέσεις ανάμεσα σε features.

Τι σημαίνουν οι τιμές:

- `pred_prob` είναι πιθανότητα positive class ή candidate success.
- Υψηλή πιθανότητα σημαίνει ότι πολλά learned tree paths δείχνουν σε θετικό outcome.
- Feature importance δείχνει ποια features χρησιμοποιήθηκαν συχνά ή με μεγάλο gain, αλλά δεν είναι causal proof.

Παράδειγμα:

- Σε meta-labeling target, `pred_prob = 0.68` δεν σημαίνει "η αγορά θα ανέβει". Σημαίνει "το candidate που έδωσε το primary signal έχει υψηλή πιθανότητα να πετύχει το target". Το σωστό signal είναι συχνά `meta_probability_side`, όχι απλό long/short threshold.

Πότε το προτιμάς:

- Όταν θες ισχυρό tabular baseline.
- Όταν περιμένεις nonlinear interactions, π.χ. momentum που δουλεύει μόνο σε συγκεκριμένο volatility regime.
- Όταν θέλεις γρήγορα πειράματα με feature importance diagnostics.

`xgboost_clf`

Τι μαθαίνει:

- Nonlinear gradient-boosted trees με XGBoost.
- Παρόμοια χρήση με LightGBM, αλλά διαφορετική υλοποίηση και regularization συμπεριφορά.

Τι σημαίνουν οι τιμές:

- `pred_prob` είναι πιθανότητα positive class.
- Πιο ακραίες τιμές δείχνουν μεγαλύτερη classifier confidence, όχι απαραίτητα καλύτερη calibration.
- Χρειάζεται calibration/threshold diagnostics πριν γίνει trading signal.

Παράδειγμα:

- Αν `pred_prob = 0.80` σε training rows αλλά μόνο `0.53` σε OOS rows, το model μάλλον υπερπροσαρμόζει. Για signal πρέπει να κοιτάς OOS distribution και όχι training confidence.

Πότε το προτιμάς:

- Όταν θέλεις boosted-tree alternative για σύγκριση με LightGBM.
- Όταν συγκεκριμένα XGBoost params ή constraints ταιριάζουν καλύτερα στο πείραμα.

Sequence και event models

`event_transformer_encoder`

Τι μαθαίνει:

- Sequence representation γύρω από candidate events.

- Χρησιμοποιεί rolling lookback sequences από features και εκπαιδεύει transformer encoder ανά time split.
- Είναι κατάλληλο για candidate-based targets, ειδικά triple-barrier ή directional/meta labels.

Τι σημαίνουν οι τιμές:

- `pred_prob`, αν ενεργοποιηθεί, είναι πιθανότητα event success.
- `event_emb_0`, `event_emb_1`, ... είναι learned embeddings, δηλαδή compressed αναπαράσταση του πρόσφατου context.
- `pred_is_oos = 1` σημαίνει ότι το event prediction/embedding προέρχεται από fold όπου το row ήταν out-of-sample.

Παράδειγμα:

- Ένα pullback candidate έχει παρόμοιο current-bar setup με άλλα, αλλά το προηγούμενο 32-bar context δείχνει επιτάχυνση volatility. Το transformer μπορεί να το κωδικοποιήσει σε embeddings και να δώσει χαμηλότερο `pred_prob` από ένα tabular one-row classifier.

Πότε το προτιμάς:

- Όταν η σειρά των προηγούμενων bars έχει νόημα και όχι μόνο τα current feature values.
- Όταν θέλεις embeddings για downstream diagnostics ή second-stage models.
- Όταν έχεις αρκετά candidates για να εκπαιδευτεί sequence model χωρίς έντονο overfit.

Return forecasters και regressors

`lightgbm_regressor`

Τι μαθαίνει:

- Nonlinear tabular regression για continuous target, συνήθως future return.
- Μπορεί να χρησιμοποιηθεί με `future_return_regression` ή άλλο continuous label.

Τι σημαίνουν οι τιμές:

- `pred_ret > 0` -> θετικό expected return.
- `pred_ret < 0` -> αρνητικό expected return.
- Το μέγεθος έχει νόημα μόνο στην κλίμακα του target. Αν το target είναι volatility-normalized, τότε και το `pred_ret` είναι σε volatility units.

Παράδειγμα:

- `pred_ret = 0.003` σε raw-return target σημαίνει expected +0.3%.
- `pred_ret = 1.2` σε volatility-normalized target σημαίνει expected move περίπου +1.2 units τοπικής μεταβλητότητας.
- Ένα `forecast_threshold` signal μπορεί να κάνει long μόνο όταν `pred_ret` ξεπερνά το κόστος και το noise threshold.

Πότε το προτιμάς:

- Όταν θες να προβλέψεις μέγεθος return, όχι μόνο αν είναι θετικό.
- Όταν θες να μετατρέψεις forecast σε risk-adjusted sizing με `forecast_vol_adjusted`.

sarimax_forecaster

Τι μαθαίνει:

- Στατιστικό time-series forecast με SARIMAX λογική.
- Είναι περισσότερο baseline για autocorrelation/seasonality structure παρά σύνθετο ML model.

Τι σημαίνουν οι τιμές:

- `pred_ret` δείχνει forecast του continuous target.
- Αν το forecast κινείται κοντά στο `0`, το μοντέλο δεν βρίσκει ισχυρή predictable structure στην κλίμακα του target.

Παράδειγμα:

- Σε ημερήσια data, αν υπάρχει εβδομαδιαίο/seasonal pattern, το SARIMAX μπορεί να δώσει μικρό θετικό `pred_ret` σε συγκεκριμένες περιόδους. Αν τα OOS metrics δεν ξεπερνούν naive baseline, δεν πρέπει να γίνει trading signal.

Πότε το προτιμάς:

- Για απλό statistical benchmark.
- Όταν θες να συγκρίνεις ML forecasters με κλασικό time-series model.

garch_forecaster

Τι μαθαίνει:

- Volatility dynamics από returns.
- Το κύριο νόημα του είναι risk/volatility forecast, όχι directional alpha.

Τι σημαίνουν οι τιμές:

- `pred_vol` υψηλό -> αναμένεται μεγαλύτερη μεταβλητότητα.
- `pred_vol` χαμηλό -> αναμένεται πιο ήρεμο περιβάλλον.
- Αν υπάρχει `pred_ret`, διάβασέ το προσεκτικά ως output της συγκεκριμένης forecasting διαδρομής. Το δυνατό σημείο του GARCH είναι το volatility.

Παράδειγμα:

- Αν δύο rows έχουν ίδιο `pred_ret = 0.002`, αλλά το πρώτο έχει `pred_vol = 0.006` και το δεύτερο `pred_vol = 0.025`, το δεύτερο έχει πολύ χειρότερο signal-to-risk ratio. Ένα volatility-adjusted signal θα δώσει μικρότερη θέση στο δεύτερο.

Πότε το προτιμάς:

- Για volatility targeting.
- Για risk filters.
- Για να τροφοδοτήσεις `forecast_vol_adjusted` ή volatility-aware sizing.

lstm_forecaster

Τι μαθαίνει:

- Sequence forecast με LSTM.
- Μαθαίνει temporal state από ordered lookback windows.

Τι σημαίνουν οι τιμές:

- `pred_ret` είναι forecast continuous target από sequence context.
- Υψηλό απόλυτο `pred_ret` σημαίνει ότι το μοντέλο βλέπει ισχυρότερο pattern στο πρόσφατο sequence, αλλά θέλει OOS έλεγχο γιατί neural models μπορούν να overfit.

Παράδειγμα:

- Αν μετά από συγκεκριμένο volatility squeeze και breakout sequence το model δίνει `pred_ret = 0.005`, αυτό σημαίνει ότι το learned recurrent state συνδέει τέτοια ακολουθία με θετική future return.

Πότε το προτιμάς:

- Όταν πιστεύεις ότι η σειρά των προηγούμενων bars έχει predictive πληροφορία.
- Όταν έχεις αρκετά δεδομένα και θέλεις neural sequence baseline.

patchtst_forecaster

Τι μαθαίνει:

- Patch-based time-series forecasting representation.
- Χωρίζει το lookback σε patches και μαθαίνει patterns σε μεγαλύτερο context.

Τι σημαίνουν οι τιμές:

- `pred_ret` διαβάζεται ως expected continuous target.
- Το model μπορεί να πιάσει patterns που δεν φαίνονται σε single-row tabular features, αλλά η αξιοπιστία κρίνεται από OOS folds.

Παράδειγμα:

- Σε 128-bar lookback, το PatchTST μπορεί να μάθει ότι συγκεκριμένη ακολουθία compression -> expansion -> shallow pullback έχει θετικό expected return. Το αποτέλεσμα εμφανίζεται ως θετικό `pred_ret`.

Πότε το προτιμάς:

- Για μεγαλύτερα sequence contexts.
- Όταν θέλεις πιο σύγχρονο neural forecasting baseline από LSTM.

tft_forecaster

Τι μαθαίνει:

- Temporal Fusion Transformer style forecasting.
- Στόχος του είναι να συνδυάζει temporal context και feature interactions σε sequence forecasting.

Τι σημαίνουν οι τιμές:

- `pred_ret` είναι forecast του continuous target.

- Αν υπάρχουν attention/feature diagnostics στο artifact, βοηθούν στην ερμηνεία, αλλά δεν αποδεικνύουν causality.

Παράδειγμα:

- Ένα row με θετικό momentum αλλά υψηλό volatility μπορεί να πάρει χαμηλότερο `pred_ret` από row με ίδιο momentum και χαμηλότερο risk, επειδή το sequence model βλέπει interaction ανάμεσα σε temporal state και risk regime.

Πότε το προτιμάς:

- Όταν θέλεις ισχυρό neural forecaster με πλουσιότερο temporal context.
- Όταν έχεις αρκετά rows και αυστηρά OOS splits.

Foundation forecasters

Τι μαθαίνουν:

- Δεν κάνουν supervised fit στο dataset. Φορτώνουν pretrained Chronos ή TimesFM checkpoint και παράγουν zero-shot forecast από ιστορικό context.
- Τα wrappers χρησιμοποιούν causal `source_col` history, όχι generated target columns, ώστε να μην διαρρεύσει future label.

Τι σημαίνουν οι τιμές:

- `pred_ret` είναι το forecast του target-horizon return που προκύπτει από το predicted price/returns path.
- `pred_qXX` είναι quantile-based return forecast όπου υποστηρίζεται από το backend.
- `pred_vol` είναι proxy uncertainty από το μισό εύρος high-low quantiles.

Πότε το προτιμάς:

- Όταν θέλεις γρήγορο zero-shot baseline χωρίς fold-level training.
- Όταν θες να συγκρίνεις Chronos-Bolt/Chronos-2 ή μικρότερα TimesFM setups με τα local neural forecasters.
- Όταν το dependency/model download είναι αποδεκτό για το runtime περιβάλλον.

Feature discovery

`tsfresh_extrema_feature_discovery`

Τι μαθαίνει:

- Δεν είναι κλασικός predictor που παράγει trading probabilities.
- Χτίζει PIT-safe rolling windows, εξάγει tsfresh features και ελέγχει ποια features σχετίζονται με future extrema labels: `neither`, `local_top`, `local_bottom`.

Τι σημαίνουν οι τιμές:

- `tsfresh_extrema_label` δείχνει αν το anchor row ήταν local top, local bottom ή neither σε future horizon.
- `tsfresh_extrema_label_code` κωδικοποιεί τα labels αριθμητικά.

- `tsfresh_extrema_eligible = true` σημαίνει ότι το row είχε πλήρες rolling feature window και future label horizon.
- Τα extracted tsfresh columns είναι candidate research features.
- Το metadata `selected_features` δείχνει ποια features πέρασαν relevance selection ανά folds.

Παράδειγμα:

- Με `window_size = 48` και `label_horizon = 8`, το component παίρνει τα 48 προηγούμενα bars ως feature window και κοιτάει τα επόμενα 8 bars για να δει αν το anchor είναι local top ή bottom. Αν ένα tsfresh feature επιλέγεται σταθερά στα folds, είναι candidate για μελλοντικό feature engineering, όχι αυτόματο production signal.

Πότε το προτιμάς:

- Για research και feature discovery.
- Όταν θέλεις να βρεις ποιες time-series ιδιότητες σχετίζονται με local extrema.
- Όταν αποδέχεσαι ότι τα selected features πρέπει μετά να ελεγχθούν σε κανονικό model/backtest workflow.

Single-asset reinforcement learning

`ppo_agent`

Τι μαθαίνει:

- Policy optimization για ένα asset με PPO.
- Μαθαίνει actions που μεγιστοποιούν reward μέσα στο trading environment, λαμβάνοντας υπόψη returns, costs, execution lag και constraints.

Τι σημαίνουν οι τιμές:

- `action_rl` είναι raw action από την policy.
- `signal_rl` είναι η μεταφρασμένη trading έκθεση.
- Θετικό `signal_rl` σημαίνει long exposure.
- Αρνητικό `signal_rl` σημαίνει short exposure, αν το environment το επιτρέπει.
- Απόλυτο μέγεθος μεγαλύτερο σημαίνει μεγαλύτερη έκθεση ή stronger action.

Παράδειγμα:

- Αν `signal_rl = 0.75`, η PPO policy ζητά 75% long exposure στην κλίμακα του environment. Αν `signal_rl = -0.40`, ζητά short 40%, εφόσον το config επιτρέπει shorting.

Πότε το προτιμάς:

- Όταν το πρόβλημα είναι sequential decision-making και όχι απλή πρόβλεψη label.
- Όταν θέλεις το model να μάθει άμεσα trade actions με costs/reward.
- Όταν έχεις αρκετά δεδομένα για robust OOS policy evaluation.

`dqn_agent`

Τι μαθαίνει:

- Discrete-action policy για ένα asset με DQN.
- Επιλέγει ανάμεσα σε διακριτές actions, π.χ. flat/long/short ή προκαθορισμένα position buckets.

Τι σημαίνουν οι τιμές:

- `action_rl` είναι το discrete action id.
- `signal_rl` είναι η έκθεση που αντιστοιχεί στο action.
- Η απόσταση μεταξύ action ids δεν σημαίνει απαραίτητα γραμμική διαφορά risk. Η σημασία ορίζεται από το action mapping του environment.

Παράδειγμα:

- Αν action `0 = flat`, `1 = long`, `2 = short`, τότε `action_rl = 1` γίνεται `signal_rl = 1`. Αν έχεις bucketed actions, π.χ. `0%`, `50%`, `100%`, τότε το action id πρέπει να διαβάζεται μέσω του mapping.

Πότε το προτιμάς:

- Όταν θέλεις discrete policy search.
- Όταν οι επιτρεπτές θέσεις είναι λίγες και ξεκάθαρες.

Portfolio reinforcement learning

`ppo_portfolio_agent`

Τι μαθαίνει:

- PPO policy για allocation ή actions σε πολλά assets.
- Λαμβάνει υπόψη portfolio constraints, gross exposure targets, asset groups και per-asset returns/features.

Τι σημαίνουν οι τιμές:

- Κάθε asset παίρνει δικό του `signal_rl`.
- Θετική τιμή σε asset σημαίνει long allocation ή long action.
- Αρνητική τιμή σημαίνει short allocation αν επιτρέπεται.
- Το σύνολο των per-asset signals πρέπει να διαβάζεται μαζί με portfolio constraints, όχι ανεξάρτητα.

Παράδειγμα:

- Για τρία assets, η policy μπορεί να γράψει `GER40 = 0.4`, `US500 = 0.3`, `XAUUSD = -0.2`. Αυτό δεν είναι τρία ανεξάρτητα signals. Είναι portfolio allocation που πρέπει να αξιολογηθεί με total gross/net exposure και group constraints.

Πότε το προτιμάς:

- Όταν η απόφαση είναι allocation μεταξύ assets.
- Όταν ενδιαφέρει correlation, diversification και group risk.

`dqn_portfolio_agent`

Τι μαθαίνει:

- Discrete-action portfolio policy.

- Επιλέγει από διακριτές portfolio actions ή allocation buckets.

Τι σημαίνουν οι τιμές:

- `action_rl` δείχνει την discrete portfolio action.
- Per-asset `signal_rl` είναι η εφαρμογή αυτής της action στα assets.
- Όπως στο single-asset DQN, το action id διαβάζεται μόνο μέσω του configured mapping.

Παράδειγμα:

- Action 3 μπορεί να σημαίνει "long equity basket, flat metals" ή άλλο predefined allocation, ανάλογα με το environment. Δεν πρέπει να συγκρίνεται ως αριθμός με action 2 χωρίς να δεις το mapping.

Πότε το προτιμάς:

- Όταν θέλεις discrete allocation regimes.
- Όταν το portfolio action space είναι μικρό και σαφές.

Παραδείγματα YAML

Classifier για meta-labeling

```
model:
  kind: lightgbm_clf
  target_col: label
  feature_selectors:
    - prefix: ema_
    - prefix: atr_
    - prefix: stoch_
  prediction_col: pred_prob
  split:
    method: walk_forward
```

Ερμηνεία:

- Το `label` πρέπει να έχει φτιαχτεί από target όπως `directional_triple_barrier`.
- Το model παράγει `pred_prob`.
- Για trading, χρησιμοποίησε μόνο rows όπου `pred_is_oos = 1`.
- Αν το primary signal έχει side/candidate, το κατάλληλο signal είναι συχνά `meta_probability_side`.

Regressor για return forecast

```
target:
  kind: future_return_regression
  params:
    price_col: close
    horizon: 8
    normalize_by_volatility: true
    volatility_col: atr_14
```

```
model:
  kind: lightgbm_regressor
  target_col: label
  prediction_col: pred_ret
  split:
    method: walk_forward
```

Ερμηνεία:

- Το `pred_ret` είναι forecast σε volatility-normalized units.
- `pred_ret = 1.0` σημαίνει περίπου μία μονάδα τοπικής μεταβλητότητας υπέρ της θετικής κατεύθυνσης.
- Για signal μπορείς να χρησιμοποιήσεις `forecast_threshold` ή `forecast_vol_adjusted`.

RL policy

```
model:
  kind: ppo_agent
  feature_cols:
    - close_logret_1
    - atr_pct_14
    - rsi_14
  params:
    window_size: 32
```

Ερμηνεία:

- Το model δεν παράγει `pred_prob`.
- Παράγει action/policy output, π.χ. `action_rl` και `signal_rl`.
- Το αποτέλεσμα αξιολογείται ως sequential strategy με costs και execution assumptions, όχι ως classifier probability.

Market-making MOMENT quote filter

Το `market_making_moment` δεν είναι canonical `MODEL_REGISTRY` entry για το candle-based pipeline. Είναι research-only market-making experiment κάτω από:

```
config/experiments/market_making/market_making_moment.yaml
```

Στόχος:

- να χτίσει quote-level/event-level dataset από `orderbook_events.csv` και `quote_events.csv`,
- να χρησιμοποιήσει MOMENT ως pretrained time-series feature extractor ή ως frozen-encoder + lightweight-head baseline,
- να σκοράρει buy/sell quote candidates ξεχωριστά,

- να εφαρμόσει fee-aware expected edge: `predicted_markout_bps + expected_spread_capture_bps - maker_fee_bps - safety_buffer_bps`,
- να αξιολογήσει αν μειώνει toxic fills και βελτιώνει fee-adjusted markout.

Παράδειγμα:

```
python scripts/run_market_making_moment_experiment.py \  
--config config/experiments/market_making/market_making_moment.yaml
```

Βασικά outputs:

- `moment_dataset.parquet`
- `moment_predictions.csv`
- `quote_decisions.csv`
- `baseline_vs_moment.csv`
- `summary.json`
- `run_metadata.json`
- `artifact_manifest.json`

Ερμηνεία των prediction columns:

- `moment_buy_score`: predicted buy-side markout/edge score.
- `moment_sell_score`: predicted sell-side markout/edge score.
- `moment_buy_expected_edge_bps`: buy score μετά από spread capture, fees και safety buffer.
- `moment_sell_expected_edge_bps`: αντίστοιχο sell-side expected edge.
- `moment_uncertainty`: uncertainty score που πρέπει να είναι κάτω από threshold.
- `moment_decision`: `allow_buy`, `allow_sell`, `allow_both` ή `block`.
- `moment_reason`: explainable reason για την απόφαση.

Leakage rules:

- Future columns όπως `future_mid_return_*`, `buy_markout_bps_*` και `sell_markout_bps_*` είναι targets/evaluation fields, όχι model inputs.
- Το split είναι chronological/walk-forward. Δεν επιτρέπεται random shuffle.
- MOMENT δεν αντιμετωπίζεται ως black-box trader. Είναι research feature extractor/filter που αξιολογείται πάνω σε local order-book data.
- Δεν επιτρέπεται demo/live order placement από αυτό το experiment.

Πρακτικός κανόνας επιλογής

- Θέλεις καθαρό baseline; ξεκίνα με `logistic_regression_clf`.
- Θέλεις regularized interpretable classifier με πολλά features; δοκίμασε `elastic_net_clf`.
- Θέλεις ισχυρό tabular classifier; σύγκρινε `lightgbm_clf` και `xgboost_clf`.
- Θέλεις continuous return forecast; χρησιμοποίησε `lightgbm_regressor` ή sequence forecaster.
- Θέλεις zero-shot foundation baseline χωρίς training; δοκίμασε `chronos_bolt_forecaster`, `chronos_2_forecaster`, `timesfm_2p5_200m_forecaster` ή `timesfm_1p0_200m_forecaster`.
- Θέλεις volatility/risk forecast; χρησιμοποίησε `garch_forecaster`.

- Θέλεις να αξιοποιήσεις temporal sequence context; δοκίμασε `event_transformer_encoder`, `lstm_forecaster`, `patchtst_forecaster` ή `tft_forecaster`.
- Θέλεις feature research για local extrema; χρησιμοποίησε `tsfresh_extrema_feature_discovery`.
- Θέλεις direct policy/action learning; χρησιμοποίησε `ppo_agent`, `dqn_agent`, `ppo_portfolio_agent` ή `dqn_portfolio_agent` ανάλογα με single asset ή portfolio setup.